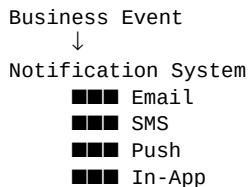


Notification System — Practical System Design Notes

Hinglish • Node.js • Queue • Redis • Email/SMS/Push • Interview-focused

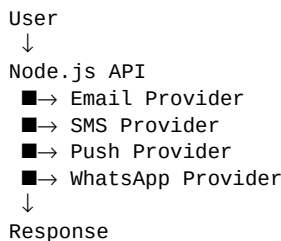
1. Notification System Kya Hai?

Notification system users ko events ke basis par messages deliver karta hai—Email, SMS, Push Notification, WhatsApp ya in-app notification ke through.



Core goal: notification ko **reliably, asynchronously, correct channel par aur duplicate ke bina** deliver karna.

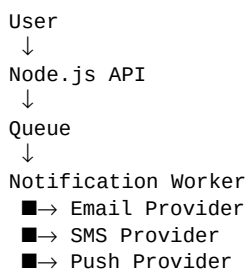
2. Direct Provider Call Problem



Problem:
slow provider → slow API
provider failure → user request affected

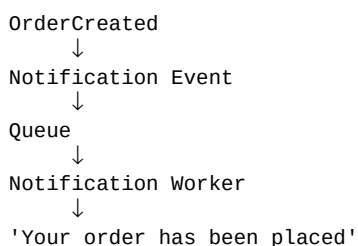
Request path mein multiple external providers call karne se latency aur failure coupling badh sakti hai.

3. Queue-Based Architecture



API notification job enqueue karke quickly response de sakti hai; worker background mein provider ko call karta hai.

4. Event-Driven Thinking



Business service ko har provider ki details jaanna zaroori nahi. Notification layer event ko appropriate channel mein translate kar sakti hai.

5. Notification Channels

Channel	Typical use
Email	Receipts, reports, account messages
SMS	OTP, urgent alerts, transactional updates
Push	Mobile/web real-time user notifications
WhatsApp	Transactional/conversational messaging where supported
In-App	Activity feed, alerts, product updates

6. Notification Job Structure

```
{
  event: "ORDER_CREATED",
  userId: "123",
  channels: ["email", "push"],
  data: {
    orderId: "ORD-1001"
  }
}
```

Actual schema application-specific hota hai. Important idea: event type + recipient + channel + payload/context.

7. Worker Architecture

```
Queue
↓
Notification Dispatcher
■→ Email Worker
■→ SMS Worker
■→ Push Worker
■→ WhatsApp Worker
```

Each worker:
→ validate
→ provider call
→ success/failure
→ retry if appropriate

8. Why Separate Workers?

Email, SMS aur Push providers ki latency, rate limits aur failure behavior different ho sakti hai. Separate workers se independent scaling aur failure isolation possible hoti hai.

```
SMS provider slow
↓
SMS workers scale/tune
↓
Email workers unaffected
```

9. Retry

Temporary provider failures, network timeout ya transient errors mein retry useful hai. Permanent errors ko blindly retry nahi karna chahiye.

```
Notification
↓
Attempt 1 → timeout
↓
backoff
↓
Attempt 2 → timeout
↓
backoff
↓
Attempt 3 → success
```

10. Exponential Backoff + Jitter

Retry delays:
1 sec
2 sec
4 sec
8 sec

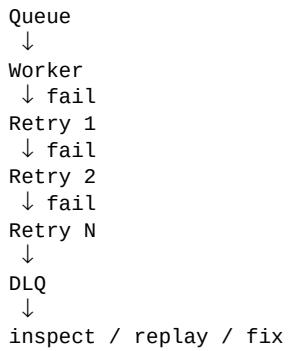
+ jitter
→ random variation

Purpose:

avoid synchronized retry storm

Maximum retry attempts define karo. Retry policy channel/provider ke according different ho sakti hai.

11. Dead Letter Queue



DLQ permanently or repeatedly failed notifications ko normal queue se separate rakhti hai, so problematic jobs processing pipeline ko block nahi karte.

12. Idempotency / Duplicate Notifications

At-least-once processing mein same job multiple times process ho sakta hai. Notification system ko duplicate sends control karne ke liye idempotency/deduplication strategy chahiye.

eventId = EVT-123

Worker attempt 1:
send email → success
worker crashes before ACK

Retry:
same EVT-123

Dedup check:
already processed?
→ YES → don't send again

13. Notification Status

Notification
↓
QUEUED
↓
PROCESSING
↓
SENT
↓
DELIVERED (if provider supports confirmation)

Failure:
PROCESSING
↓
RETRYING
↓
FAILED / DLQ

SENT aur DELIVERED same nahi hote. Provider accepted the message vs recipient device/mailbox actually received it are different states.

14. Provider Failure

Email Provider A ■

Possible:
Queue retry
↓
Provider A again

or configured fallback:
Provider B
↓
Email sent

Fallback blindly duplicate send create kar sakta hai. Provider switching ko idempotency, provider response semantics aur business rules ke saath design karna chahiye.

15. Multi-Channel Fallback

Critical alert

Push
↓ unavailable?
SMS
↓ unavailable?
Email

But:
channel preference

cost
urgency
user consent
must be considered.

Har notification ko automatically all channels par send karna correct nahi. User preferences and notification policy important hain.

16. User Notification Preferences

User 123

Marketing:

Email ☒

SMS ☒

Push ☒

Transactional:

Email ☒

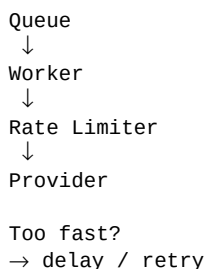
SMS ☒

Push ☒

Transactional/security notifications aur marketing notifications ke rules generally different ho sakte hain. Compliance and consent requirements region/provider/business ke according apply hoti hain.

17. Rate Limiting Providers

Email/SMS/Push providers per-second, per-minute, account-level or cost-based limits impose kar sakte hain. Notification workers ko provider limits respect karni chahiye.



18. Priority Notifications

Critical OTP/security alert aur low-priority marketing email ko same queue treatment dena ideal nahi ho sakta.

High Priority Queue
→ OTP
→ Security Alert

Normal Queue
→ Order update

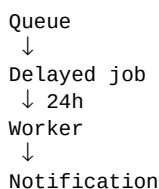
Low Priority Queue
→ Marketing

Separate queues ya priority scheduling use karke critical notifications ko faster processing mil sakti hai.

19. Delayed Notifications

Event:
Order placed

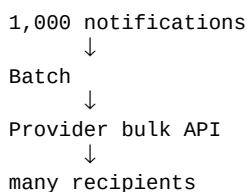
Schedule:
'Send reminder after 24 hours'



Delayed jobs queue technology ke according different ways se implement ho sakte hain. BullMQ delayed jobs ka common Node.js example hai.

20. Batching

High-volume notifications ko individual provider calls ke bajay batching/bulk APIs ke through process karna provider support aur use case ke according more efficient ho sakta hai.



21. Backpressure

Agar events 50,000/sec aa rahe hain aur provider sirf 5,000/sec handle karta hai, workers ko blindly 50,000 calls/sec nahi karni chahiye. Queue backlog + controlled concurrency + rate limiting required hai.

Incoming:



Provider capacity:



Queue absorbs temporary spike

+

workers process at safe rate

22. Notification Deduplication

Kabhi same business event multiple times publish ho sakta hai. Event ID ya deterministic idempotency key se duplicate notifications detect ki ja sakti hain.

```
dedupe key:
ORDER_CREATED:ORD-1001:user-123:email
```

```
If key already processed:
→ skip duplicate
```

23. Template Service

Notification content ko code ke andar hard-code karne ke bajay template layer use ki ja sakti hai. Template variables runtime data se populate hote hain.

```
Template:
"Hi {{name}}, your order {{orderId}}
has been shipped."
```

```
Data:
name = Sonu
orderId = ORD-1001
```

24. Localization

Multi-language users ke liye template + locale selection useful hai. Notification payload mein locale/user language preference include ki ja sakti hai.

```
user.locale = "hi-IN"
```

```
Event
↓
Template selector
↓
Hindi template
↓
Provider
```

25. Notification Service API

POST /notifications

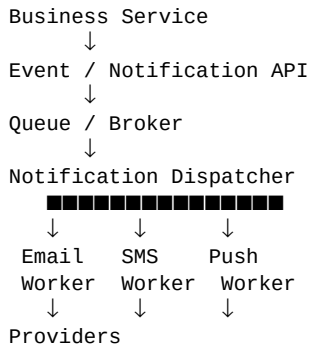
```
{
  "userId": "123",
  "type": "ORDER_CREATED",
  "channels": ["push", "email"],
  "data": {
    "orderId": "ORD-1001"
  }
}
```

API:
validate → authorize → enqueue → response

26. Sync vs Async Notification

Approach	Use
Synchronous	Rare cases where immediate provider result is required
Asynchronous	Transactional/background notifications, high volume, retries
Hybrid	Critical immediate response + async secondary channels

27. Notification System + Queue



28. Notification System + Redis

Redis can support rate limiting, short-lived deduplication state, delayed-job infrastructure in some systems, caching of templates/config, and other fast state. It should not automatically be treated as the permanent source of notification history.

29. Notification History

Notification History

id
user_id
event_id
channel
status
provider_message_id
created_at
sent_at
failed_at

Purpose:

- audit/debug
- user notification center
- delivery tracking

Long-term history ko durable storage mein rakhna generally appropriate hota hai; exact store depends on scale and product requirements.

30. Observability

- Notifications queued/sec
- Queue depth
- Oldest notification age
- Processing latency
- Provider latency
- Success/failure rate by channel
- Retry count
- DLQ count
- 429/provider rate-limit responses
- Duplicate/deduplication count
- Delivery rate where provider supports delivery events

31. Production Architecture

Users



Load Balancer



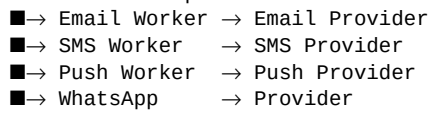
Node.js APIs × N



Event / Queue



Notification Dispatcher



Supporting:

Redis → rate limit / dedupe / fast state

Durable store → notification history

Monitoring → metrics + logs + traces

DLQ → failed jobs

32. Failure Scenarios

Failure	Possible handling
Provider timeout	Retry with exponential backoff
Provider rate limit	Delay/throttle and retry
Permanent invalid recipient	Mark failed; avoid endless retries
Worker crash	Message can become available again depending on queue semantics
Duplicate job	Idempotency/deduplication
Queue outage	Controlled failure/retry/outbox depending on criticality
Provider outage	Retry, fallback provider where safe

33. Interview Q&A;

Question	Short answer
Notification system kya karta hai?	Events ko user-facing messages mein convert karke channels par deliver karta hai.
Queue kyun?	Request latency reduce, provider failure isolate, retry and scaling.
Worker?	Background process that consumes notification jobs.
Idempotency?	Same job multiple times process hone par duplicate side effect avoid karna.
DLQ?	Repeatedly failed notifications ke liye failure queue.
SENT vs DELIVERED?	Provider accepted/sent vs recipient actually received; exact semantics provider-specific.
Provider down?	Retry/backoff; safe fallback where appropriate.
Why separate workers?	Independent scaling, rate limits and failure isolation.
Why rate limiting?	Provider quotas and system protection.
Priority queue?	Critical notifications ko normal/low priority se faster process karna.
Delayed notification?	Job ko future time par process karna.
Notification history?	Audit, debugging, delivery tracking and user notification center.
Why Redis?	Fast shared state for rate limits/dedupe/config/cache; exact role depends on design.

34. Practice Scenarios

1. Order placement ke baad email + push bhejna hai. Design the system.
2. Email provider 10 minutes down hai. What happens?
3. Same notification job twice arrive ho gaya. Duplicate email kaise prevent karoge?
4. SMS provider 100 requests/sec allow karta hai but queue 10,000 jobs/sec receive kar rahi hai. Design handling.
5. OTP ko marketing email se higher priority kyun doge?
6. Worker notification send karne ke baad crash ho gaya before ACK. What can happen?
7. User ne push notifications disable ki hain. System kya kare?
8. Notification provider says message accepted, but user hasn't received it. SENT vs DELIVERED explain karo.
9. Design fallback: Push → SMS → Email. Duplicate risks kaise handle karoge?
10. How would you monitor the notification system in Grafana?
11. How would you support Hindi/English templates?
12. Design a notification system for 1 million notifications/hour.

35. Final Cheat Sheet

NOTIFICATION SYSTEM CHEAT SHEET

Event
↓
Queue
↓
Worker
↓
Channel Provider

Channels:
Email / SMS / Push / WhatsApp / In-App

Must handle:
→ retry
→ backoff + jitter
→ DLQ
→ idempotency
→ deduplication
→ rate limits
→ priority
→ delayed jobs
→ preferences
→ templates/localization
→ status tracking
→ observability

Important:
SENT ≠ DELIVERED
At-least-once → duplicates possible
Provider failure → retry/fallback
Critical channels → stricter policy

Golden interview line:
"I'll keep notification delivery asynchronous, use channel-specific workers, enforce provider rate limits, implement retries with backoff, idempotency/DLQ for failures, user preferences, and monitor queue and delivery metrics."

36. Local Practice

Node.js + BullMQ se notification system banao: ORDER_CREATED event enqueue karo, Email/SMS/Push workers create karo, fake providers ke failures simulate karo, retry + DLQ implement karo, eventId-based idempotency add karo, provider rate limiting add karo, aur Grafana/Prometheus mein queue depth, notification latency, retries aur failures monitor karo.